4.1 Console

Inside any controller action or view, you can invoke the console by calling the console method. For example, in a controller:

Or in a view:

This will render a console inside your view. You don't need to care about the location of the console call; it won't be rendered on the spot of its invocation but next to your HTML content.

The console executes pure Ruby code: You can define and instantiate custom classes, create new models and inspect variables.

Only one console can be rendered per request. Otherwise web-console will raise an error on the second console invocation.

4.2 Inspecting Variables

You can invoke instance_variables to list all the instance variables available in your context. If you want to list all the local variables, you can do that with local_variables.

4.3 Settings

config.web_console.whitelisted_ips: Authorized list of IPv4 or IPv6 addresses and networks (defaults: 127.0.0.1/8, ::1).
config.web_console.whiny_requests: Log a message when a console rendering is prevented (defaults: true).

Since web-console evaluates plain Ruby code remotely on the server, don't try to use it in production. 5 Debugging Memory Leaks

A Ruby application (on Rails or not), can leak memory — either in the Ruby code or at the C code level. In this section, you will learn how to find and fix such leaks by using tool such as Valgrind.

5.1 Valgrind